SQL AGGREGATE FUNCTIONS & OPERATIONS
EXERCISE 2

1. LIST ALL DISTINCT DEPARTMENT IN THE STUDENT TABLE

```
SELECT DISTINCT department
FROM            student;
```

department
IT
HR
Finance

2. GET THE AVERAGE AGE OF STUDENT PER DEPARTMENT

```
SELECT department, AVG(age) AS avg_age
FROM   student
GROUP BY department;
```

department	avg_age
IT	20,5
HR	22
Finance	23

$IT = \frac{20+21}{2} = 20{,}5$

$HR = \frac{22+22}{2} = 22$

3. SHOW DEPARTMENT WITH MORE THAN 1 STUDENT

```
SELECT department, COUNT(*) AS student_count
FROM   student
GROUP BY department
HAVING COUNT (*) > 1;
```

department	student_count
IT	2
HR	2

4. GET ALL STUDENTS WHOSE AGE IS BETWEEN 21 AND 23

```
SELECT student_id, name, age, department
FROM students
WHERE age >= 21 AND age <= 23;
```

student_id	name	age	department
3	Charlie	21	IT
4	Diana	23	Finance
2	Bob	22	HR
5	Eve	22	HR

5. LIST ALL STUDENTS IN THE IT OR HR DEPARTMENT WHO ARE OLDER THAN 21

```
SELECT student_id, name, age, department
FROM students
WHERE department = 'IT' or department = 'HR' AND age >21;
```

Student_id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT
5	Eve	22	HR

6. SHOW TOTAL CREDITS PER DEPARTMENT, ONLY FOR DEPARTMENT WITH MORE THAN 5 TOTAL CREDITS

```
SELECT department, SUM(credits) AS total_credits
FROM courses
GROUP ~~WHERE~~ BY department
HAVING SUM(credits) > 5;
```

course_id	course_name	department	credits
101	SQL Basics	IT	3
102	PYTHON	IT	4
103	Data Science	IT	4

7. LIST ALL COURSES THAT DO NOT HAVE 4 CREDITS

SELECT course-id, course-name, department, credits

FROM courses

WHERE credits ! = 4;

course-id	course-name	department	credits
101	SQL Basics	IT	3
104	Excel	Finance	2
105	statistics	HR	3

8. SHOW THE TOP 3 COURSES BY CREDITS IN DESCENDING ORDER

SELECT ~~*~~ course-id, course-name, credits

FROM courses

ORDER BY credits DESC

LIMIT 3;

course-id	course-name	credits
~~105~~ ~~102~~ 101	~~Statistics IT~~ ~~Python~~ SQL Basic	3
102	Python	4
103	Data Science	4

9. GET THE MAXIMUM, MINIMUM, AND AVERAGE GRADE ACROSS ALL ENROLMENTS

SELECT MIN(grade) AS min-grade, MAX(grade) AS max-grade, AVG(grade) AS avg-gr ~~min-grade, max-grade, avg-grade~~

FROM enrolments;

max-grade	90
min-grade	78
avg-grade	84.6

10. COUNT HOW MANY ENROLMENTS EXIST PER COURSE

SELECT course-id, COUNT(*) AS enrollment ~~course~~-count

FROM enrolments;
GROUP BY course-id;

course-id	enrollment_count
101	1
102	2
103	3
104	4
105	5

11. FIND TOTAL SALARY AND TOTAL BONUS PER DEPARTMENT

SELECT department, SUM (salary) AS total_salary, SUM(bonus) AS total_bonus
FROM ~~em~~ salaries_table
GROUP BY department;

department	total_salary	total_bonus
HR	109 000	7500
Finance	70 000	6000
IT	122 000	10 500

12. SHOW DEPARTMENTS WHERE AVERAGE SALARY IS ABOVE 55 000

SELECT department, AVG (salary) AS average-salary
FROM employees
GROUP BY department
HAVING AVG (salary) > 55 000

department	avg_salary
IT	60 000
IT	62 000

13. LIST EMPLOYEES WHOSE SALARY PLUS BONUS IS GREATER THAN 60 000.

SELECT employee-id, name, salary, bonus, (salary + bonus) AS total-compensation

FROM salaries-table

WHERE (salary + bonus) > 60000;

employee-id	name	salary	bonus	total-compensation
1	Tom	60000	5000	65000
3	Spike	70000	6000	76 000
4	Tyko	62000	5500	67 500

14. SHOW TOTAL AND AVERAGE BUDGET PER DEPARTMENT. ONLY INCLUDE DEPARTMENT WITH AVERAGE BUDGET ABOVE 70 000.

SELECT department, SUM(budget) AS total-budget, AVG(budget) AS avg-budget

FROM projects

GROUP BY department

HAVING AVG(budget) > 70000

department	total-budget	avg-budget
IT	270 000	135 000
Finance	80 000	80 000

15. LIST ALL PROJECTS WITH BUDGETS BETWEEN 50 000 AND 120000, EXCLUDING THE MARKETING DEPARTMENT.

SELECT project-id, project-name, department, budget

FROM projects

WHERE budget BETWEEN 50000 AND 120 000 AND department <> 'marketing';

project-id	project-name	department	budget
1	AI App	IT	120000
2	Payroll System	Finance	80000
3	Dashboard	IT	150000
5	HR Portal	HR	50000